

Local Setup — Yuno

For the live demo session. Three commands from clone to a working orchestration platform with two LLM providers, a Telegram bot, and five demo workflows seeded.

0. Prerequisites

	Required	How to check
Docker Desktop	≥ 20.10 with Compose v2	<code>docker --version && docker compose version</code>
RAM available to Docker	2 GB minimum (4 GB comfortable)	Docker Desktop → Settings → Resources
Disk	~3 GB free	<code>df -h .</code>
Python 3	3.10+ (host, for the port-scanning helper)	<code>python3 --version</code>
<code>make</code>	any	<code>make --version</code>
One LLM API key (optional)	Anthropic, OpenAI, or Gemini	Yuno runs in <code>LLM_MODE=stub</code> (deterministic canned replies) without keys, so the <i>flow</i> still demos offline. Set <code>LLM_MODE=live</code> + a key for real output.

Nothing else to install. Everything else (Postgres + pgvector, Redis, Python deps, Node deps, Caddy-style routing) ships inside containers.

1. Three-command setup

```
git clone <repo>                                # whatever your clone URL is
cd yuno-orchestrator

cp .env.example .env                             # optional: open .env and add keys
make up                                           # builds everything; auto-picks free host
make seed                                         # Jarvis + standing team + 5 demo templates
```

That's it. `make up` runs `scripts/dev-up.sh`, which:

- Scans for free host ports (so it works even if you already have something on 3000/8000)
- Boots `postgres` → `redis` → `backend` → `worker` → `frontend` → `code-runner` → `playwright-mcp`

- Auto-launches the host-side **Claude Code bridge** (for the optional on-machine coding feature)
- Prints the URLs at the end:

```
UI      http://localhost:3001      (or next free port – usually 3000)
API     http://localhost:8000     (OpenAPI docs at /docs)
```

`make seed` is **idempotent** — safe to re-run. Creates only what's missing.

Expected timing

	First run	After that
<code>make up</code>	5–10 min (image builds — Python deps + Next.js deps)	~20 s
<code>make seed</code>	< 5 s	< 5 s
Frontend cold-start in browser	~10 s (Next.js dev compile on first hit per route)	instant

2. Verification checklist

After `make seed`, in this exact order:

	Check	What you should see
1	<code>curl http://localhost:8000/health</code>	<code>{"status":"ok","env":"dev","llm_mode":"live"}</code> (or <code>stub</code>)
2	Open the UI (<code>http://localhost:3001</code>)	Cockpit "Mission Control" with the Jarvis constellation (15 named specialists ringed around JARVIS)
3	Cockpit gauges	<code>AGENTS 17</code> , <code>SPEND \$...</code> , <code>TOKENS ...</code> — non- — values mean stats API works
4	<code>/agents</code>	Search bar + 17 rows with model badges (claude / gpt / gemini), one Run + Chat + Edit per row
5	<code>/workflows</code>	5 templates: Market Briefing, Draft & Critique, Cited Research (tools badge), Page Summariser · MCP (mcp badge), PRD with Approval (approval badge)
6	<code>/team</code>	Three channels: <code>#growth</code> , <code>#product</code> , <code>#research</code> with named members
7	<code>/channels</code>	<code>Telegram · @<your_bot></code> shown with an <code>ACTIVE</code> pill (if <code>TELEGRAM_BOT_TOKEN</code> set)
8	<code>docker compose exec backend pytest -q</code>	<code>84 passed, 1 skipped</code>

If any check fails, see **§5 troubleshooting**.

3. The demo path

A 2–3 minute walk-through that touches every brief requirement. Run in this order:

Step 1 — Cockpit (the system at a glance, ~15 s)

- Land on `/` . Point at:
 - The **gauges** (live cost + token counters)
 - The **constellation** — "15 named specialists, one chief of staff (Jarvis)"
 - The **mission queue** below — every prior run as history

Step 2 — Multi-agent workflow with a tool node (~45 s) — *the brief's "execute real tools"*

- Navigate to `/workflows` , click ► **Run on Cited Research (demo)** .
- Enter a topic, e.g. `"prompt caching for LLM agents – benefits and pitfalls"` .
- Watch the timeline:

- **search** step — the `web_search` tool node runs first (status badge, no LLM cost — it's a tool, not an agent)
- **cite → analyse → brief** — three agents in series, each step shows the agent's name, tokens, cost, and the actual output
- End on a brief that cites real, current sources (proves Tavily was hit live).

Step 3 — MCP integration (~30 s) — *the brief's "configurable tools"*

- Back to `/workflows`, click ► Run on **Page Summariser · MCP (demo)**.
- Paste any URL.
- Two MCP Playwright steps (`browser_navigate` → `browser_snapshot`) followed by Brie summarising. **Point out** that adding browser capability required *zero backend code* — it's all MCP.

Step 4 — Human-in-the-loop (~30 s) — *the brief's "interaction rules"*

- `/workflows` → ► Run on **PRD with Approval (demo)**.
- Enter an idea, e.g. `"a Slack bot that summarises long threads"`.
- Pip drafts a PRD → run **pauses on the approval gate** (a red pill appears on the cockpit's `APPROVALS` gauge).
- Click ✓ Approve → Mara + Brie finish.

Step 5 — Async agent-to-agent + Telegram (~30 s) — *the brief's "agent-to-agent + external channel"*

- Open Telegram → message your bot (`@<your_bot>` from `/channels`).
- It reaches **Jarvis** (chief of staff), which may delegate to specialists via `send_message_to_agent` — each delegation appears as a **child run** linked from the parent.
- Reply lands back in Telegram.
- Show the same conversation on `/runs/<id>` — every handoff, every tool call, full cost ledger.

Step 6 — Cost guardrail (~15 s) — *the brief's "limits"*

- On the agent's edit page, set `max_cost_per_run_usd: 0.02`.
- Run a workflow expensive enough to trip it. The run terminates gracefully with a **destructive-coloured timeline block** and `runs.error = "cost cap exceeded"`.

Total: ~2:45.

4. Configuration knobs you'll likely touch

In `.env` :

Var	Purpose	Default
<code>LLM_MODE</code>	<code>stub</code> (offline, free) · <code>live</code> (real providers via ModelRouter)	<code>stub</code>
<code>ANTHROPIC_API_KEY</code> / <code>OPENAI_API_KEY</code> / <code>GEMINI_API_KEY</code>	Provider keys — set at least one for <code>live</code> mode	empty
<code>TAVILY_API_KEY</code>	Real <code>web_search</code> tool. Without it, <code>web_search</code> returns a graceful stub.	empty
<code>TELEGRAM_BOT_TOKEN</code>	Your bot (get one from @BotFather in 2 min)	empty
<code>TELEGRAM_TRANSPORT</code>	<code>polling</code> (no TLS needed, default) or <code>webhook</code>	<code>polling</code>
<code>PUBLIC_BASE_URL</code>	Only needed for <code>webhook</code> transport — a public HTTPS URL pointed at the backend	empty

After editing `.env`, restart only what needs it:

```
docker compose restart backend worker
```

5. Troubleshooting

Symptom	Cause	Fix
<code>make up</code> fails on <code>next build</code> -step	Docker has < 2 GB RAM	Docker Desktop → Settings → Resources → bump Memory to 4 GB → restart
Cockpit gauges stuck on <code>-</code>	Stats API not responding	<code>`docker compose logs backend`</code>
Frontend takes 17 s on first page hit	Next.js cold-compile in dev mode	Normal once per route. Subsequent hits are instant. To eliminate, run in production mode (see §6).
<code>Cited Research</code> returns no real sources	<code>TAVILY_API_KEY</code> missing	Add it to <code>.env</code> and <code>docker compose restart worker</code> — or accept the stub result for the demo.
Telegram bot doesn't respond	Token is already polling from another machine	Telegram allows only ONE poller per bot. Stop the other instance, or create a second bot via @BotFather.
<code>make seed</code> shows existing agents but no new ones	Already seeded — idempotent	This is the success path. Nothing to do.
Jarvis names agents that don't exist	Stale long-term memory in pgvector	<code>docker compose exec backend python -m scripts.cleanup_demo_state</code> — purges orphan memories + ghost team channels. Re-runs are no-ops.
Pytest fails on <code>test_shared_memory_is_recalled_with_attribution</code>	Pre-existing test isolation bug — old <code>codename%</code> memories crowd out the new probe	<code>docker compose exec postgres psql -U yuno -d yuno -c "DELETE FROM memories WHERE content ILIKE '%codename%' OR content ILIKE '%spring release%';"</code> then re-run pytest.

6. Reset / re-seed / nuke

```
# Soft reset – clear test artefacts (junk agents, ghost workflows, stale memories), k
docker compose exec backend python -m scripts.cleanup_demo_state

# Re-seed (idempotent – only creates what's missing)
make seed

# Hard reset – destroy DB volume and start over
docker compose down -v
make up
make seed
```

To stop everything cleanly: `make down`. Containers + the host Claude bridge come down together.

7. (Optional) Production-mode frontend

`make up` runs the frontend in `next dev` for HMR. For a snappier demo without the 10-s first-hit compile:

```
docker compose exec frontend npm run build
docker compose exec frontend npm run start
```

Drops idle RAM from ~740 MB → ~200 MB and removes cold-start latency on every route. Suitable for the recorded demo.

8. Where to read more

- `docs/architecture.md` — full architecture rationale (two-layer execution model, harness, channels, memory, guardrails)
- `docs/plan.md` — build plan with rationale per subsystem
- `README.md` — quickstart + feature catalogue + scope/cut table
- `backend/scripts/cleanup_demo_state.py` — exactly what the cleanup script does
- `backend/scripts/seed.py` — the canonical agent + workflow + team-channel definitions